

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:
Гордеев Артём
К3339

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

Реализация REST API на основе boilerplate с использованием стека Go (Gin + GORM + PostgreSQL).

Сайт для поиска работы

1. Вход
2. Регистрация
3. Личный кабинет пользователя (с резюме)
4. Поиск вакансий с фильтрацией по отрасли, зарплате, опыту
5. Страница с деталями вакансии (описание, требования, компания)
6. Личный кабинет работодателя (управление вакансиями)

Ход работы

1 Стек технологий

Для реализации REST API был выбран следующий стек:

- Язык: Go 1.26
- HTTP-фреймворк: Gin - высокопроизводительный веб-фреймворк для Go
- ORM: GORM - объектно-реляционный маппер с поддержкой миграций
- База данных: PostgreSQL 16
- Аутентификация: JWT (JSON Web Tokens) - библиотека golang-jwt
- Хэширование паролей: bcrypt
- Запуск БД: Docker Compose

2 Структура проекта

```
lab1/
├── cmd/api/           # Точка входа приложения (main.go)
├── internal/
│   ├── config/        # Загрузка конфигурации из .env
│   ├── database/      # Подключение к PostgreSQL через GORM
│   ├── handlers/      # Обработчики HTTP-запросов (бизнес-логика)
│   │   ├── auth.go    # Регистрация, вход, выход, смена пароля
│   │   ├── profile.go  # Управление профилями соискателя и работодателя
│   │   ├── company.go  # CRUD для компаний
│   │   ├── job.go      # CRUD для вакансий, поиск и фильтрация
│   │   ├── resume.go   # CRUD для резюме (навыки, образование, опыт)
│   │   ├── application.go # Отклики на вакансии
│   │   └── dictionary.go # Справочник отраслей
│   ├── middleware/    # JWT-аутентификация и RBAC (разграничение ролей)
│   ├── models/        # GORM-модели базы данных
│   ├── routes/        # Конфигурация маршрутов Gin
│   └── utils/         # Утилиты (генерация/валидация JWT-токенов)
├── docker-compose.yml # Оркестрация PostgreSQL
└── .env              # Переменные окружения
```

3 Схема базы данных

Схема была спроектирована в ДЗ1 и доработана с учётом замечаний: навыки, образование и опыт работы вынесены в отдельные таблицы.

Итоговая реляционная модель включает следующие таблицы:

- users - базовые данные для аутентификации (email, password_hash, role)
- job_seekers - профиль соискателя (first_name, last_name, phone)
- employers - профиль работодателя (first_name, last_name, position, company_id)
- companies - компании-работодатели (name, description, website)
- industries - справочник отраслей (id, name)
- jobs - вакансии (title, description, requirements, salary_min, salary_max, experience, status)
- resumes - резюме соискателей (title, summary, salary_expectation, is_active)
- skills - навыки соискателя (name, resume_id)
- educations - образование (institution, degree, start_date, end_date, resume_id) - отдельная таблица
- experiences - опыт работы (company_name, position, description, start_date, end_date, resume_id)
- job_applications - отклики на вакансии (job_id, job_seeker_id, resume_id, cover_letter, status)

4 Реализованные эндпоинты

Аутентификация

Метод	Путь	Описание
POST	/auth/register	Регистрация пользователя (job_seeker / employer)
POST	/auth/login	Вход, получение JWT-токена
GET	/auth/me	Информация о текущем пользователе
POST	/auth/change-password	Смена пароля
POST	/auth/logout	Выход (токен аннулируется на

		клиенте)
--	--	----------

Профили

Метод	Путь	Описание	Роль
GET	/profile/job-seeker	Просмотр профиля соискателя	job_seeker
PUT	/profile/job-seeker	Обновление профиля соискателя	job_seeker
GET	/profile/employer	Просмотр профиля работодателя	employer
PUT	/profile/employer	Обновление профиля работодателя	employer

Компании

Метод	Путь	Описание	Роль
POST	/companies	Создание компаний	employer
GET	/companies/:id	Просмотр компаний	публичный
PUT	/companies/:id	Обновление компаний	employer

Вакансии

Метод	Путь	Описание	Роль
GET	/jobs	Список вакансий (поиск и фильтрация)	публичный
GET	/jobs/:id	Детали вакансии	публичный
POST	/jobs	Создание вакансии	employer
PUT	/jobs/:id	Обновление вакансии	employer
DELETE	/jobs/:id	Архивирование вакансии	employer

Поддерживаемые параметры фильтрации для GET /jobs:

- query - полнотекстовый поиск по названию и описанию
- experience - фильтр по требуемому опыту
- salary_expected - фильтр по ожидаемой зарплате (пересечение с вилкой)

Резюме

Метод	Путь	Описание	Роль
POST	/resumes	Создание резюме (с навыками, образованием, опытом)	job_seeker
GET	/resumes	Список своих резюме	job_seeker
GET	/resumes/:id	Просмотр резюме	публичный
PUT	/resumes/:id	Обновление резюме	job_seeker
DELETE	/resumes/:id	Удаление резюме	job_seeker

Отклики на вакансии

Метод	Путь	Описание	Роль
POST	/jobs/:id/apply	Отклик на вакансию	job_seeker
GET	/applications/me	История откликов соискателя	job_seeker
DELETE	/applications/:id	Удаление своего отклика	job_seeker
GET	/jobs/:id/applications	Отклики на вакансию	employer
PATCH	/applications/:id/status	Изменение статуса отклика	employer

Справочники

Метод	Путь	Описание
GET	/industries	Список отраслей

5 Безопасность

- Пароли хранятся в виде bcrypt-хэша
- Аутентификация реализована через JWT Bearer-токены
- Разграничение доступа по ролям (RBAC) реализовано через middleware:
- AuthMiddleware - проверяет наличие и валидность JWT
- RoleMiddleware - проверяет роль пользователя (job_seeker / employer)
- Защита от IDOR: перед изменением объекта проверяется принадлежность его владельцу
- Запрет дублирующих откликов на уровне уникального индекса БД

6 Запуск приложения

```
# 1. Запустить PostgreSQL через Docker Compose
docker compose up -d

# 2. Настроить .env
# DB_HOST=localhost, DB_USER=postgres, DB_PASSWORD=postgres
# DB_NAME=jobboard, DB_PORT=5432
# JWT_SECRET=supersecretkey, PORT=8080

# 3. Запустить API-сервер
go run cmd/api/main.go
```

7 Тестирование

Тестирование функциональности всех эндпоинтов проводилось с помощью Postman. Подробные результаты тестирования можно увидеть в ДЗ3: Тестирование API средствами Postman.

Покрытие тестами включает:

- Аутентификацию: регистрацию двух ролей, вход, смену пароля, проверку токена, выход
- Профили соискателя и работодателя: получение и обновление; проверку разграничения ролей
- Компании: создание, получение, обновление; проверку запрета создания для соискателей
- Вакансии: полный CRUD; фильтрацию по опыту, зарплате и тексту
- Резюме: полный CRUD с вложенными объектами (навыки, образование, опыт)
- Отклики: отправку, просмотр (соискатель и работодатель), смену статуса, защиту от дублей, удаление

Вывод

В ходе лабораторной работы был реализован полнофункциональный REST API для сервиса по поиску работы на стеке Go + Gin + GORM + PostgreSQL. Реализована чистая архитектура с разделением на слои: маршруты, обработчики, модели и утилиты. Применена JWT-аутентификация с разграничением ролей (RBAC). Схема базы данных доработана относительно ДЗ1 и ДЗ2: навыки, образование и опыт вынесены в отдельные таблицы для соответствия требованиям нормализации. Сервис полностью протестирован через Postman, все 29 эндпоинтов функционируют корректно.